Phase 3: GenAI / RAG Pipeline — Complete Learning Guide

Table of Contents

---

1. The Big Picture

WHERE WE ARE

```
Phase 1 (DONE): Raw data → Clean validated data
Phase 2 (DONE): Clean data → ML predictions + metrics
Phase 3 (YOU ARE HERE): Predictions → Plain English Q&A
Phase 4 (NEXT): Everything → API + Dashboard
```

WHAT PHASE 3 ADDS

Phases 1-2 produced CSVs and pickle files. Useful for data scientists,
useless for supply chain managers who don't know Python.

Phase 3 adds a NATURAL LANGUAGE INTERFACE:

WITHOUT Phase 3:
Manager: "Which parts might run out?"
Analyst: "Let me open Python, load the CSV, filter by trend..."
(20 minutes later)
Analyst: "AMAT-RF-7800-GEN shows increasing demand."

WITH Phase 3:
Manager types: "Which parts are at stockout risk?"
System (instantly): "AMAT-RF-7800-GEN shows 25% demand increase.
MAPE is 54.6%, indicating high forecast uncertainty. Recommend
increasing safety stock by 15%."

THE KEY INSIGHT: WHY NOT JUST USE ChatGPT?

If you paste your question into ChatGPT without your data, it will HALLUCINATE — make up numbers that sound right but aren't based on anything real. It doesn't know YOUR parts, YOUR forecasts, YOUR trends.

RAG solves this by:

1. FIRST retrieving YOUR actual data
2. THEN generating an answer GROUNDED in that data
3. CITING the specific sources it used

The AI becomes a researcher who reads YOUR documents before answering, not a guesser who makes things up.

---

2. The 6 Core Concepts (from zero)

CONCEPT 1: EMBEDDINGS — GPS coordinates for meaning

WHAT: A list of numbers (vector) that represents the MEANING of text.

ANALOGY: Imagine every sentence has a GPS coordinate.
"The dog ran fast" → location (3.2, 7.1, 5.8, ...)
"The puppy sprinted quickly" → location (3.3, 7.0, 5.9, ...) ← CLOSE!
"Stock prices fell" → location (91.0, 2.5, -3.1, ...) ← FAR AWAY

Same meaning = close coordinates. Different meaning = far coordinates.

HOW IT WORKS TECHNICALLY:
A neural network (Sentence Transformer) reads your text word by word. The last hidden layer outputs 384 numbers. These 384 numbers ARE the embedding. The model was trained on millions of sentence pairs to learn that similar sentences should produce similar number lists.

WHY 384 NUMBERS, NOT 2?
Two numbers (like GPS) can only capture 2 aspects of meaning.
384 numbers can capture 384 aspects: is it about technology? is it positive? is it about manufacturing? is it urgent?
More dimensions = more nuanced understanding.

INTERVIEW ANSWER:
"Embeddings convert text into dense numerical vectors where semantic similarity maps to vector proximity. I used the all-MiniLM-L6-v2 Sentence Transformer model which produces 384-dimensional embeddings.

This enables semantic search where 'stockout risk' matches data about 'increasing demand with high forecast uncertainty' despite zero keyword overlap."

CONCEPT 2: VECTOR DATABASE (ChromaDB) — Google Maps for meaning

WHAT: A database that finds things by MEANING SIMILARITY instead of exact keyword matching.

REGULAR DATABASE:
Query: WHERE part_number = 'AMAT-RF-7800-GEN'
Only works if you know the exact part number.

VECTOR DATABASE:
Query: "parts with supply risk"
Finds AMAT-RF-7800-GEN because its description mentions "increasing demand" and "high MAPE" — semantically close to "supply risk" even without those exact words.

HOW IT WORKS:

1. Every data chunk gets embedded (converted to a vector)
2. ChromaDB stores the text + its vector
3. When you search, your query gets embedded too
4. ChromaDB calculates the DISTANCE between your query vector and every stored vector
5. Returns the closest matches (smallest distance = most similar)

THE MATH — COSINE SIMILARITY:
Two vectors are similar if they point in the same direction.
Cosine similarity = cos(angle between them)
= 1.0 if identical direction (same meaning)
= 0.0 if perpendicular (unrelated)
= -1.0 if opposite (opposite meaning)

WHY CHROMADB:

- Free and open source
- Runs locally (no cloud account needed)
- Designed for RAG applications
- Persistent storage (data survives restarts)
- Simple Python API

INTERVIEW ANSWER:
"I used ChromaDB as the vector store for its lightweight local

deployment and persistent storage. The knowledge base contains 13 documents covering part profiles, model metrics, and domain knowledge. Semantic search retrieves the top-3 most relevant chunks per query using cosine similarity, enabling meaning-based matching rather than keyword dependence."

CONCEPT 3: CHUNKING — Breaking data into searchable pieces

WHAT: Splitting your data into small, self-contained text pieces that each contain ONE complete idea.

WHY NOT EMBED THE WHOLE CSV?

1. Embedding models have token limits (typically 512 tokens)
2. If you embed a 1000-row CSV as one chunk, the embedding captures a vague average of everything — not specific enough to match specific questions
3. The LLM context window is limited — you can't paste 1000 rows

OUR CHUNKING STRATEGY:
One chunk per part = one complete profile including:

- Part number and type
- Demand statistics (average, min, max, trend)
- Forecast accuracy (MAE, MAPE)
- Stockout risk assessment
- Recommendation

WHY THIS WORKS:
When the user asks about AMAT-RF-7800-GEN, the search retrieves that part's COMPLETE profile — not just its MAE without context, not just its trend without the recommendation.

CONCEPT 4: PROMPT TEMPLATE — The exam instructions

WHAT: A structured instruction that tells the LLM how to use the retrieved data.

OUR TEMPLATE:

```
You are an expert supply chain analyst.
Answer ONLY based on the context data below.
If the data doesn't contain the answer, say so.
Always cite specific numbers from the context.

CONTEXT: {data retrieved from ChromaDB}
QUESTION: {what the user asked}
```

WHY "ONLY based on context":
This is the anti-hallucination instruction. Without it, the LLM might mix its general knowledge with your specific data, producing answers that SOUND authoritative but contain made-up numbers.

WHY "cite specific numbers":
Forces the LLM to ground its answer in actual data. "Demand is increasing" is vague. "Demand increased 25%, from rolling average 6.2 to 8.3 units" is grounded and verifiable.

CONCEPT 5: LANGCHAIN — The plumbing

WHAT: A Python library that connects embedding models, vector databases, prompt templates, and LLMs into a single pipeline.

WITHOUT LANGCHAIN (manual plumbing):

python

```python
embedding = sentence_transformer.encode(question)
results = chromadb.query(embedding, top_k=3)
context = "\n".join(results.documents)
prompt = template.format(context=context, question=question)
answer = openai.chat(prompt)
```

WITH LANGCHAIN:

python

```python
chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4"),
    retriever=chromadb.as_retriever(),
    chain_type="stuff",
)
answer = chain.run(question)
```

Both do the same thing. LangChain saves you from writing the plumbing manually. In our project, we implement the manual version (SimpleRAG) so you understand every step, and provide the LangChain version (FullRAG) for production.

CONCEPT 6: AGENTS — The smart assistant with a toolbox

WHAT: An AI that DECIDES which tools to use based on the question, instead of following a fixed script.

REGULAR RAG: Always does the same 3 steps (embed → search → generate).

AGENT RAG: Reads the question, thinks about what tools it needs:

- "Which parts are at risk?" → search tool (vector search)
- "What's the forecast for AMAT-RF-7800-GEN?" → direct lookup tool
- "Compare NPI vs standard parts" → search tool TWICE, then compare
- "What model performed best?" → metrics lookup tool

The agent REASONS about the right approach. This is the "agentic" part of our project name.

---

3. File 1: knowledge_base.py — The Library Builder

WHAT IT DOES

Takes your Phase 2 outputs (predictions, metrics, features) and converts them into 13 searchable text chunks organized into 3 categories:

```
8 part profile chunks     — one per semiconductor part
1 model summary chunk     — XGBoost vs ARIMA comparison
4 domain knowledge chunks — NPI, standard, service campaign, reliability
```

CODE WALKTHROUGH — CHUNKING STRATEGY

The most important function is create_part_profile_chunks(). It reads the ML features CSV and creates a natural language description for each part:

```
python
```

```
text = f"""Part {part} is a {demand_type}-type semiconductor component
primarily serving the {region} region.

Demand profile: Average daily demand is {avg_demand} units
(range: {min_demand} to {max_demand} units)...

Stockout risk assessment: {stockout_risk}

Recommendation: {recommendation}"""
```

TECHNICAL — WHY TEXT, NOT STRUCTURED DATA:
LLMs understand natural language far better than structured formats.

BAD (structured): {"part": "AMAT-RF-7800-GEN", "mae": 2.10, "mape": 54.6}
The LLM might confuse mae with mape, or not understand what 2.10 means.

GOOD (narrated): "Part AMAT-RF-7800-GEN achieves MAE of 2.10 units
and MAPE of 54.6%."
The LLM understands the complete sentence and can reason about it.

BUSINESS — WHY INCLUDE STOCKOUT RISK ASSESSMENT:
We pre-compute a risk rating (HIGH/MODERATE/LOW) based on:

- Is demand trending up? (increasing pressure)
- Is MAPE high? (uncertain forecasts)
- Is volatility high? (unpredictable demand)

This way, the RAG can directly answer "which parts are at risk?"
without the LLM having to compute risk from raw numbers.

CODE WALKTHROUGH — METADATA

Each chunk has metadata attached:

python

```
chunks.append({
    "id": "part_AMAT-RF-7800-GEN",
    "text": "Part AMAT-RF-7800-GEN is a NPI-type...",
    "metadata": {
        "part_number": "AMAT-RF-7800-GEN",
        "demand_type": "NPI",
        "region": "North America",
        "stockout_risk": "HIGH",
        "source": "part_profile",
    },
})
```

TECHNICAL — WHY METADATA MATTERS:
Metadata enables FILTERED search. Instead of searching all 13 chunks,
you can filter to only "source=part_profile" or "demand_type=NPI."
This makes search faster and more accurate.

HOW TO RUN

```bash
python -m src.genai.knowledge_base
```

EXPECTED OUTPUT:

```
Building RAG Knowledge Base
  Part profiles: 8 chunks
  Model summaries: 1 chunks
  Domain knowledge: 4 chunks
  Total: 13 chunks
  ChromaDB: Active (or "JSON fallback" if not installed)
```

---

4. File 2: rag_pipeline.py — The Question Answerer

TWO MODES

SimpleRAG (always works):

- Keyword matching for search
- Template-based answer generation
- No external APIs needed

- Good for development and learning

FullRAG (production quality):

- Semantic search via ChromaDB embeddings
- LLM generation via GPT-4
- Requires: pip install chromadb sentence-transformers openai
- Requires: OPENAI_API_KEY in .env

CODE WALKTHROUGH — SIMPLE SEARCH

```python
def search(self, query, top_k=3):
    query_words = set(query.lower().split())
    stop_words = {"the", "a", "is", "what", "which", ...}
    query_words = query_words - stop_words
```

TECHNICAL: We remove stop words because they appear in EVERY document and don't help distinguish relevant from irrelevant. "Which parts are at risk" → after removing stop words → {"parts", "risk"}.

```python
    for chunk in self.chunks:
        matches = sum(1 for word in query_words if word in combined)
        if query.lower() in combined:
            matches += 3  # Bonus for exact phrase match
```

TECHNICAL: Each chunk gets a relevance score based on how many query words appear in it. Exact phrase matches get a bonus of 3 because they're stronger signals than individual word matches.

CODE WALKTHROUGH — QUESTION TYPE DETECTION

```python
if any(word in query_lower for word in ["risk", "stockout", "shortage"]):
    return self._answer_risk_question(context_chunks, query)
elif any(word in query_lower for word in ["forecast", "predict", "mape"]):
    return self._answer_forecast_question(context_chunks, query)
```

TECHNICAL: Without an LLM, we detect the question TYPE by looking for keywords, then use a specialized answer template for each type. This

is a simple form of intent classification.

WITH AN LLM: You skip this entirely. The LLM reads the context + question and generates a natural, contextual answer.

CODE WALKTHROUGH — FULL RAG SEMANTIC SEARCH

```python
results = self.collection.query(
    query_texts=[query],
    n_results=top_k,
)
```

TECHNICAL: ChromaDB's .query() method:

1. Embeds the query text using the same model as the stored documents
2. Calculates cosine similarity against all stored vectors
3. Returns the top_k closest matches with their distances

The distance tells you HOW relevant each result is:

- distance = 0.0 → identical meaning
- distance < 0.5 → very relevant
- distance > 1.0 → weakly related

HOW TO RUN

```bash
python -m src.genai.rag_pipeline
```

EXPECTED OUTPUT — 5 sample questions answered with sources:

```
Q: Which parts are at stockout risk?
  HIGH RISK: AMAT-RF-7800-GEN (NPI, trend: increasing)
  ...
  Sources: part_AMAT-RF-7800-GEN, part_AMAT-RF-7800-CTRL
```

---

5. How to Execute Everything

```bash
```

```
# Make sure Phase 1 and Phase 2 outputs exist first
# (ml_features.csv, xgboost_predictions.csv, model_comparison.csv)

# Step 1: Build the knowledge base
python -m src.genai.knowledge_base

# Step 2: Run the RAG demo (5 sample questions)
python -m src.genai.rag_pipeline

# Optional: Install GenAI libraries for full semantic search
pip install chromadb sentence-transformers openai

# Optional: Set your OpenAI API key
# Add to .env file: OPENAI_API_KEY=sk-your-key-here

# Then re-run to use full RAG:
python -m src.genai.knowledge_base    # Re-builds with ChromaDB
python -m src.genai.rag_pipeline      # Now uses semantic search + GPT-4
```

WHAT TO CHECK

```
Step 1 → knowledge_base.json exists (13 chunks)
Step 1 → If ChromaDB installed: data/chromadb/ directory created
Step 2 → 5 questions answered with specific data citations
Step 2 → Sources listed for each answer (transparency)
```

---

6. SimpleRAG vs FullRAG — What Changes

	SimpleRAG	FullRAG
Search method	Keyword matching	Cosine similarity
Search quality	Exact words only	Understands meaning
Answer generation	Templates	GPT-4 natural language
Dependencies	None (just Python)	chromadb, openai
API key needed	No	Yes (OpenAI)
Cost	Free	~$0.01 per query
Production ready	Demo only	Yes
Hallucination risk	Zero (templates)	Low (constrained prompt)
Answer quality	Structured, rigid	Natural, contextual

The ARCHITECTURE is identical. The only difference is the quality of search (semantic vs keyword) and generation (LLM vs template).

---

7. Metrics & Testing for RAG

WHAT TO MEASURE

Retrieval precision: Out of 3 retrieved chunks, how many were relevant?
Target: > 80%
Test: Ask "Tell me about AMAT-RF-7800-GEN" — all 3 results should be about that specific part or NPI parts.

Retrieval recall: Were all relevant chunks retrieved?
Target: > 70%
Test: Ask "Which NPI parts exist?" — should retrieve all 3 NPI parts.

Answer faithfulness: Does the answer match the source data?
Target: > 90%
Test: Check that every number in the answer appears in the sources.

Hallucination rate: Does the answer contain made-up information?
Target: < 5%
Test: Verify no numbers or claims appear that aren't in the sources.

Response time: How fast is the answer?
Target: < 2 seconds
Test: Time the query() function.

---

8. Interview Preparation

QUESTION 1: "Explain RAG and why you used it."

YOUR ANSWER:
"RAG stands for Retrieval-Augmented Generation. Instead of relying on an LLM's training data, which might be outdated or generic, RAG first retrieves relevant data from a private knowledge base, then generates an answer grounded in that data. I used RAG because our supply chain data is proprietary and changes daily — the LLM needs current, company-specific context to give useful answers. My pipeline achieves this by embedding forecast data and metrics into ChromaDB, then using semantic search to find the most relevant chunks for each query before passing them to GPT-4 with a constrained prompt."

QUESTION 2: "How do you prevent hallucination?"

YOUR ANSWER:
"Three mechanisms. First, the prompt template explicitly instructs the LLM to answer ONLY from the provided context and to say 'I don't have enough data' when the context is insufficient. Second, I include specific numbers in the context chunks — MAE, MAPE, demand averages — so the LLM cites real data instead of generating plausible-sounding numbers. Third, every answer includes source attribution showing which chunks were used, enabling verification."

QUESTION 3: "Why not just put all data in the prompt?"

YOUR ANSWER:
"Two reasons: token limits and noise. GPT-4 has a 128K token context window, but our knowledge base could be much larger in production. More importantly, stuffing irrelevant data into the prompt REDUCES answer quality because the LLM has to filter signal from noise. RAG solves both problems — it retrieves only the 3-5 most relevant chunks, keeping the prompt focused and within token limits."

QUESTION 4: "What embedding model did you use and why?"

YOUR ANSWER:
"all-MiniLM-L6-v2, a 384-dimensional Sentence Transformer model. I chose it because it balances quality and speed — it's small enough to run locally without a GPU but produces high-quality embeddings for semantic similarity tasks. For production with larger knowledge bases, I'd consider OpenAI's text-embedding-3-small for its superior multilingual support, or a fine-tuned domain-specific model trained on semiconductor supply chain terminology."

QUESTION 5: "What would you improve?"

YOUR ANSWER:
"Four things. First, hybrid search combining semantic search with keyword matching (BM25) for better retrieval. Second, a reranker model that re-scores the top-20 results to pick the best 3-5 — this significantly improves precision. Third, evaluation using frameworks like RAGAS to systematically measure retrieval quality, faithfulness, and answer relevance. Fourth, a feedback loop where users can thumbs-up or thumbs-down answers, which feeds back into chunk quality improvement."